Introduction to Haskell Programming

Data structures – Queues

S P Suresh

Chennai Mathematical Institute

Haskell @ CMI

A Queue module

- In a stack, elements are pushed and popped at the top

A Queue module

- In a stack, elements are pushed and popped at the top
- In a queue, elements are added at the rear and removed from the head

A Queue module

- In a stack, elements are pushed and popped at the top
- In a queue, elements are added at the rear and removed from the head
- The Queue module, saved in Queue.hs

```
module Queue(Queue, enqueue, dequeue, empty, isEmpty) where
data Queue a = Queue [a] deriving Eq
enqueue x (Queue xs) = Queue (xs++[x])
dequeue (Queue (x:xs)) = (x, Queue xs)
empty = Queue []
isEmpty (Queue xs) = null xs
```

A Queue module

- Each `enqueue` on a queue of length n takes $O(n)$ time

A Queue module

- Each `enqueue` on a queue of length n takes $O(n)$ time
- Enqueueing and dequeueing n elements might take $O(n^2)$ time

A `Queue` module

- Each `enqueue` on a queue of length n takes $O(n)$ time
- Enqueueing and dequeueing n elements might take $O(n^2)$ time
- A more efficient queue can be built by using two lists `front` and `back`

A Queue module

- Each `enqueue` on a queue of length n takes $O(n)$ time
- Enqueueing and dequeueing n elements might take $O(n^2)$ time
- A more efficient queue can be built by using two lists `front` and `back`
- `queue = front ++ reverse back`

A Queue module

- Each `enqueue` on a queue of length n takes $O(n)$ time
- Enqueueing and dequeueing n elements might take $O(n^2)$ time
- A more efficient queue can be built by using two lists `front` and `back`
- `queue = front ++ reverse back`
- To enqueue, add an element to the head of `back`

A Queue module

- Each `enqueue` on a queue of length n takes $O(n)$ time
- Enqueueing and dequeueing n elements might take $O(n^2)$ time
- A more efficient queue can be built by using two lists `front` and `back`
- `queue = front ++ reverse back`
- To enqueue, add an element to the head of `back`
- To dequeue, remove an element from the head of `front`

A Queue module

- Each `enqueue` on a queue of length n takes $O(n)$ time
- Enqueueing and dequeueing n elements might take $O(n^2)$ time
- A more efficient queue can be built by using two lists `front` and `back`
- `queue = front ++ reverse back`
- To enqueue, add an element to the head of `back`
- To dequeue, remove an element from the head of `front`
  - What if `front` is empty?

A Queue module

- Each `enqueue` on a queue of length n takes $O(n)$ time
- Enqueueing and dequeueing n elements might take $O(n^2)$ time
- A more efficient queue can be built by using two lists `front` and `back`
- `queue = front ++ reverse back`
- To enqueue, add an element to the head of `back`
- To dequeue, remove an element from the head of `front`
  - What if `front` is empty?
  - Reverse `back` into `front` and dequeue

A Queue module

- Efficient queue

```
module Queue(Queue, enqueue, dequeue, empty, isEmpty, fromList, toList) where

data Queue a = Queue [a] [a] deriving Eq
instance Show a ⇒ Show (Queue a) where
    show = show . toList

fromList l = Queue (l, [])
toList (Queue f b) = f ++ reverse b
```

A Queue module

- Efficient queue

```
module Queue(Queue, enqueue, dequeue, empty, isEmpty,
        fromList, toList) where ...
enqueue x (Queue f b)       = Queue f (x:b)

dequeue (Queue [] [])       = error "Empty queue"
dequeue (Queue [] b)        = (x, Queue f []) where
                (x:f)       = reverse b
dequeue (Queue (x:f) b)     = (x, Queue f b)

empty                       = Queue [] []
isEmpty (Queue f b)         = null f && null b
```

Amortized analysis

- If we add n elements, we get a queue

```
Queue [] [qn,q(n-1), ... ,q1]
```

Amortized analysis

- If we add n elements, we get a queue

```
Queue [] [qn,q(n-1), ... ,q1]
```

- The next dequeue takes $O(n)$ time to reverse the list

Amortized analysis

- If we add n elements, we get a queue

```
Queue [] [qn,q(n-1), ... ,q1]
```

- The next dequeue takes $O(n)$ time to reverse the list
- After one dequeue we get:

```
Queue [q2, ... ,qn] []
```

Amortized analysis

- If we add `n` elements, we get a queue

```
Queue [] [qn,q(n-1), ... ,q1]
```

- The next dequeue takes $O(n)$ time to reverse the list
- After one dequeue we get:

```
Queue [q2, ... ,qn] []
```

- Next `n-1` dequeue operations take $O(1)$ time

Amortized analysis

- How many times is an element touched?

Amortized analysis

- How many times is an element touched?
  - Once when it is added to the second list

Amortized analysis

- How many times is an element touched?
  - Once when it is added to the second list
  - Twice when it is moved from the second to first

Amortized analysis

- How many times is an element touched?
  - Once when it is added to the second list
  - Twice when it is moved from the second to first
  - Once when it is removed from the first list

Amortized analysis

- How many times is an element touched?
  - Once when it is added to the second list
  - Twice when it is moved from the second to first
  - Once when it is removed from the first list
- Each element is touched at most four times

Amortized analysis

- How many times is an element touched?
  - Once when it is added to the second list
  - Twice when it is moved from the second to first
  - Once when it is removed from the first list
- Each element is touched at most four times
- Any sequence of n instructions involves at most n elements

Amortized analysis

- How many times is an element touched?
  - Once when it is added to the second list
  - Twice when it is moved from the second to first
  - Once when it is removed from the first list
- Each element is touched at most four times
- Any sequence of n instructions involves at most n elements
- So any sequence of n instructions takes only $O(n)$ steps

Amortized analysis

- How many times is an element touched?
  - Once when it is added to the second list
  - Twice when it is moved from the second to first
  - Once when it is removed from the first list
- Each element is touched at most four times
- Any sequence of n instructions involves at most n elements
- So any sequence of n instructions takes only $O(n)$ steps
- Queues are also very fundamental – an application in the next class!

Applying queues – an ancient Telugu riddle

15 brahmins and 15 thieves had to spend a dark night at an isolated temple of **Durga**. At midnight, the Goddess appeared in person and wanted to devour just 15 persons because She was hungry. The thieves naturally suggested that She should consume the 15 soft-limbed brahmins. But the brahmins proposed that all the 30 would stand in a circle and that Durga should eat each ninth person. The proposal was accepted by Durga and the thieves. So the brahmins arranged themselves and the thieves in a circle, telling each one where to stand. Durga counted out each ninth person and devoured him. When 15 were thus eaten, She was satiated and disappeared, and only brahmins now remained in the circle.

How do you arrange the brahmins and thieves in the circle?

The Vanadurga riddle – history

- Most probably translated from Sanskrit to Telugu

The Vanadurga riddle – history

- Most probably translated from Sanskrit to Telugu
- Not clear when it was composed or who the author is

The Vanadurga riddle – history

- Most probably translated from Sanskrit to Telugu
- Not clear when it was composed or who the author is
- Check **Mathematical Literature in Telugu: an overview** for a more detailed account

The Vanadurga riddle – history

- Most probably translated from Sanskrit to Telugu
- Not clear when it was composed or who the author is
- Check **Mathematical Literature in Telugu: an overview** for a more detailed account
- Solution provided in verse!

The Vanadurga riddle – solution

- How to implement a circle with a “current position” as a queue

The Vanadurga riddle – solution

- How to implement a circle with a “current position” as a queue
- The queue should list elements of the circle in clockwise order

The Vanadurga riddle – solution

- How to implement a circle with a “current position” as a queue
- The queue should list elements of the circle in clockwise order
- Starting from the element at the current position

The Vanadurga riddle – solution

- How to implement a circle with a “current position” as a queue
- The queue should list elements of the circle in clockwise order
- Starting from the element at the current position
- Shifting the current position to the right amounts to the following

The Vanadurga riddle – solution

- How to implement a circle with a “current position” as a queue
- The queue should list elements of the circle in clockwise order
- Starting from the element at the current position
- Shifting the current position to the right amounts to the following
  - Dequeuing the first element and enqueueing it

The Vanadurga riddle – solution

- How to implement a circle with a “current position” as a queue
- The queue should list elements of the circle in clockwise order
- Starting from the element at the current position
- Shifting the current position to the right amounts to the following
  - Dequeuing the first element and enqueueing it
  - Previous position is the last element from the new current position

The Vanadurga riddle – solution

- How to implement a circle with a “current position” as a queue
- The queue should list elements of the circle in clockwise order
- Starting from the element at the current position
- Shifting the current position to the right amounts to the following
  - Dequeuing the first element and enqueueing it
  - Previous position is the last element from the new current position
- Killing an element amounts to dequeueing the first element

The Vanadurga riddle – solution

- How to implement a circle with a “current position” as a queue
- The queue should list elements of the circle in clockwise order
- Starting from the element at the current position
- Shifting the current position to the right amounts to the following
  - Dequeuing the first element and enqueueing it
  - Previous position is the last element from the new current position
- Killing an element amounts to dequeueing the first element
- Use these ideas to arrive at a solution

The Vanadurga riddle – solution

- What if we move one step clockwise? What is the list representing the new configuration?

The Vanadurga riddle – solution

- What if we move one step clockwise? What is the list representing the new configuration?
- Instead of the moving the position ahead, move the list behind

The Vanadurga riddle – solution

- What if we move one step clockwise? What is the list representing the new configuration?
- Instead of the moving the position ahead, move the list behind
- The previous head is now the last element

```
moveRight (x:xs) = xs ++ [x]
```

The Vanadurga riddle – solution

- What if we move one step clockwise? What is the list representing the new configuration?
- Instead of the moving the position ahead, move the list behind
- The previous head is now the last element

```
moveRight (x:xs) = xs ++ [x]
```

- Can use efficient queues to avoid the costly `(++)` operator

```
moveRight q = let (x,q') = dequeue q in enqueue q' x
```

The Vanadurga riddle – full solution

```
import Queue
-- Assume m ⩾ 2, r < n, r ⩾ 0. Here (m,r,n) = (9,15,30)
vanadurga m r n = kill m r n (fromList [1..n], empty)
kill m r n (surv, dead)
    | r == 0    = (surv, dead)
    | otherwise = kill m (r-1) (n-1)
                  (shift (m-1 `mod` n) (surv, dead))
shift n (surv,dead)
    | n == 0        = (surv', enqueue x dead)
    | otherwise     = shift (n-1) (enqueue x surv', dead)
    where (x,surv') = dequeue surv
```